Benchmarking DAOS Filesystem on Aurora

Rebecca George [1,2] Cissie Mei [2] Jie Ren [1]

[1]William & Mary [2]Jefferson Lab

DAOS Storage System

Distributed Asynchronous Object Storage (DAOS) is a high performance storage system using key-value-array store allowing end-to-end user space operation, asynchronous I/O operation, and non-blocking I/O.

DAOS is provisioned to users in pools, which are persistent storage allocations spanning a set of DAOS storage engines, with a certain amount of tier 0 (Storage Class Memory (SCM)) and tier 1 (NVMe) storage. Any value or array value extent above a certain size (default=4 KiB) gets written to Tier 1, otherwise it gets written to Tier 0.

Users may create containers in their pool to store data. DAOS containers are logical collections of objects, which can be presented in a variety of formats, such as POSIX, HDF5, SQL Database, or PyDAOS. Containers' redundancy factor (number of replicates) may be set to some number n to provide tolerance for n engine failures. Containers may also be erasure coded and can use different object classes to manipulate sharding, redundancy, and erasure coding of data.

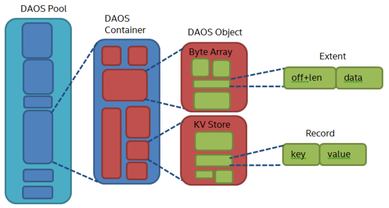

Figure 1. Diagram of DAOS pool data heirarchy [2]

DAOS can be accessed via the native DAOS library libdaos, mounting a container via DAOS FUSE, and the interception library libpil4dfs. The native DAOS libary will provide the best performance, while the interception library provides the same perfromance for I/O operations without code changes. Metadata operations via dfuse are slower than using the DFS API as the DFS API bypasses the kernel entirely.

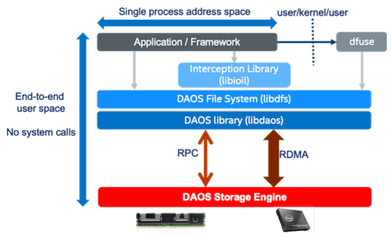

Figure 2. Diagram of DAOS storage system communication flow from applications to DAOS storage engines [2]

Aurora Architecture

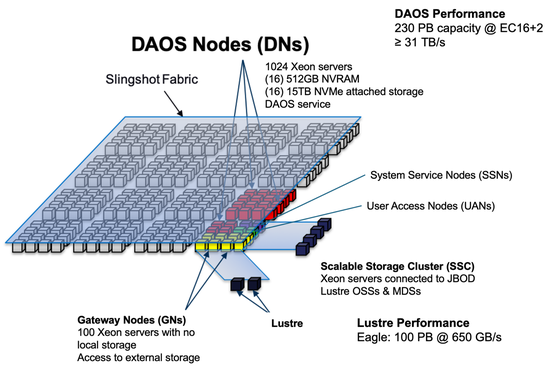

Figure 3. Diagram of DAOS storage system architecture on Aurora from Argonne National Lab [1]

Our pool had 127 nodes (4064 targets, 32 targets per node) with 3 TB Tier 0 (SCM) storage and 97 TB Tier 1 (NVMe) storage. Aurora compute nodes use two Intel Xeon CPU Max Series, with 52 cores each. For ior and mdtest we bind to 48 cores per node.

Benchmarking

We evaluated this storage system using the fio, ior, and mdtest I/O benchmarks. In particular we are interested on the effects of client-side scaling, amount of data transferred at a time (block size for fio, transfer size for ior), number of processes per node, and access type (read/write, random/sequential) on latency, IOPS, and bandwidth.

fio Bandwidth

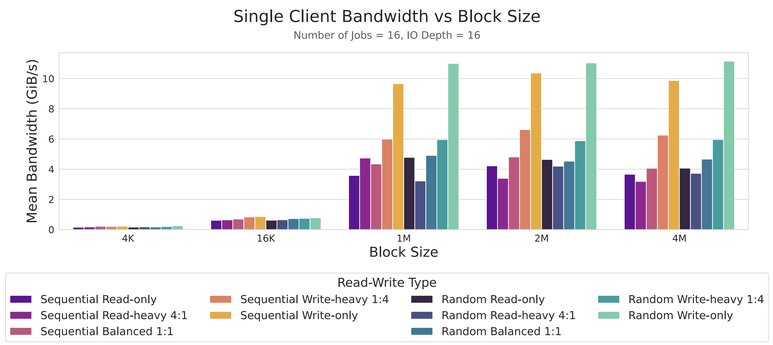

Figure 4. For one client node with 16 jobs with 16 I/O units kept in flight, the bandwidth vs block size for fio using the pvsync I/O engine. Block size in fio is the size of an I/O unit. This desmonstrates the performance achieved by POSIX container access via dfuse mount points. pvsync I/O engine uses vectorized system calls and allows read/write from non-contiguous buffers.

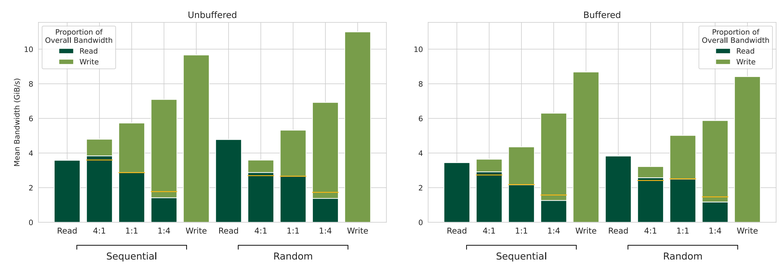

Figure 5. For one client node with 16 jobs with 16 I/O units kept in flight, the proportions of overall bandwidth spent reading and writing across different workloads. The yellow line indicates the expected ratio of bandwidth proportions. Buffered I/O will use an intermediate buffer between applications (fio) and system calls to preadv*/pwritev*, while unbuffered I/O will go straight to system calls.

We did not observe a large difference in performance between random and sequential access patterns. Additionally, for dfuse access, changing the number of fio jobs (additional threads cloning the original thread) or I/O depth (number of I/O units to keep in flight) has little effect on performance, while the number of jobs using the native DFS API increases latency (Fig. 6) and bandwidth (Fig. 7,??).

IOR Latency

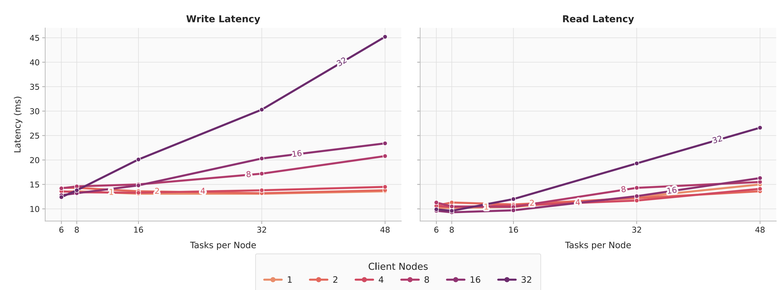

Figure 6. For a transfer size of 1 MiB, the ior latency vs number of tasks. For <8 nodes, the write latency stays relatively constant as the number of tasks increases, but larger numbers of nodes accrue more latency as the number of tasks increases. Read latency ellicited a <10 ms increase in latency as more tasks were used, but all test cases stayed under 20 ms latency (10 ms difference from 1 task) except the largest number of nodes used (32).

IOR Bandwidth

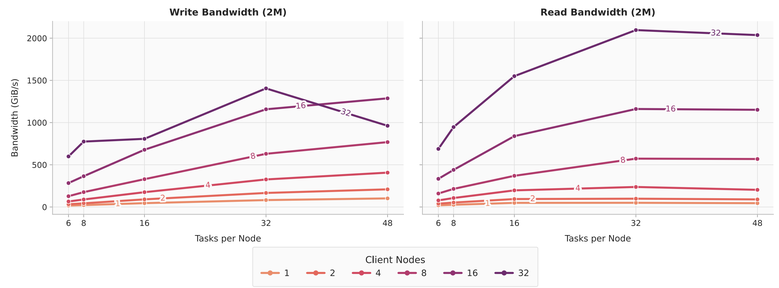

Figure 7. For a transfer size of 2 MiB, the ior bandwidth vs number of tasks, using the DFS API

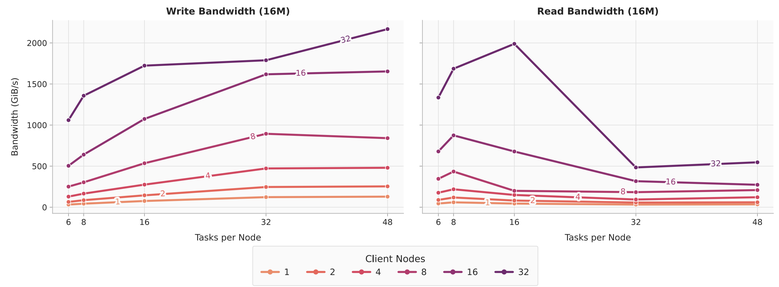

Figure 8. For a transfer size of 16 MiB, the ior bandwidth vs number of tasks, using the DFS API.

With a large transfer size, we see that reading with more than 8 or 16 tasks per node decreases bandwidth dramatically, although smaller numbers of tasks provide higher bandwidth than the same number of tasks run with a smaller transfer size. A small transfer size, such as 16 KiB provided relatively lower bandwidths than either 1-2 MiB or 16 MiB, due to having to make many small I/O operations. Reading has similar throughput to writing using the native DFS API, until the number of nodes used become larger, when it surpassed write bandwidth.

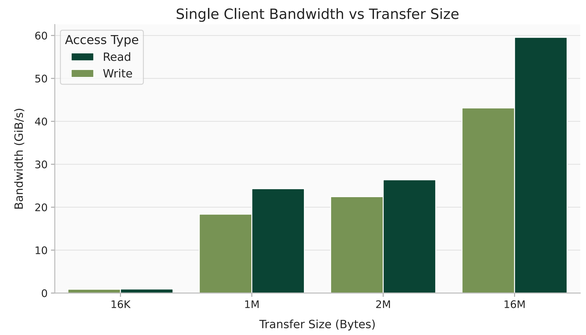

Figure 9. For a transfer size of 1 MiB, the ior bandwidth vs number of tasks, using the DFS API.

As with the pvsync I/O engine, the DFS API begins to saturate at transfer sizes above 1-2 MiB. The same transfer sizes, however, provide 2-3x higher bandwidth for single-client workloads. Unlike the non-native API, we see read bandwidths are on a similar scale to write bandwidth and are higher overall, instead of being dramatically lower than write bandwidths.

Acknowledgments

This research used resources of the Argonne Leadership Computing Facility, which is a U.S. Department of Energy Office of Science User Facility operated under contract DE-AC02-06CH11357.

References

[1] Argonne National Lab. Daos - alcf user guides, 2025. URL `https://docs.alcf.anl.gov/aurora/data-management/daos/daos-overview/`.

[2] DAOS Project. Welcome to daos version 2.6! - daos v2.6, 2016. URL `https://docs.daos.io/latest/`.